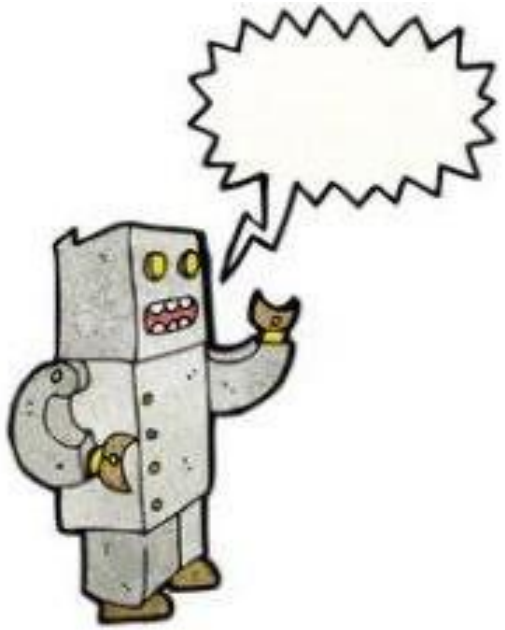
CSCI218: Foundations of Artificial Intelligence

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

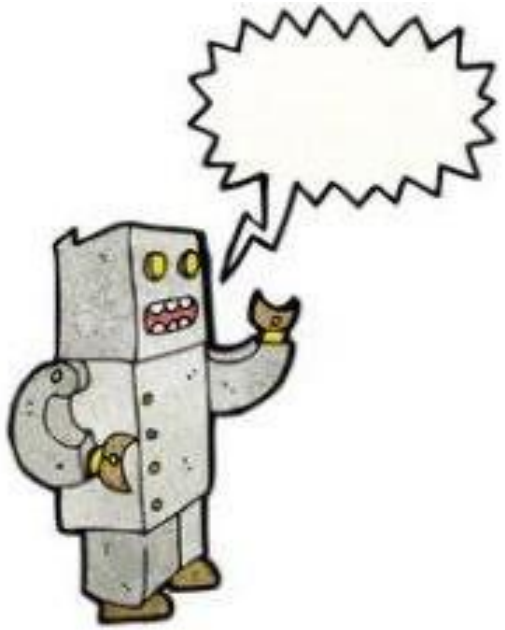
Natural Language Processing

- Complications of Real Natural Language
- Language models
- The bag-of-words model
- Naïve Bayes Classifier
- N-gram word models
- Part-of-speech (POS) tagging
- Natural Language Tasks
- **Deep Learning for Natural Language Processing**



Word Embedding

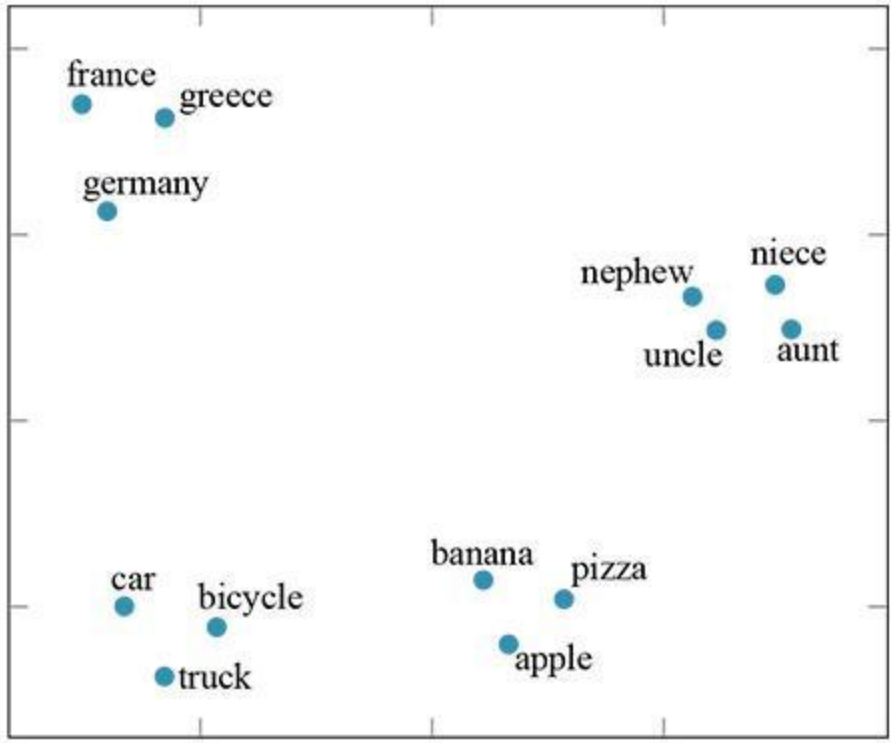
- How should we encode a word into a vector
- One-hot coding
  - encode the ith word in the dictionary with a “1” in the ith position and a “0” in all the other positions.
  - It would not capture the similarity between words.
  - “cat” vs “kitten”; “car” vs “automobile”
- We need a representation that allow for
  - **Generalization** between related words



Word Embedding

- Let's learn this representation automatically from the data!
- How does it look like?

$$
\begin{aligned}
\text{\textbackslash\textbackslash aardvark//} &= [-0.7, +0.2, -3.2, \ldots] \\
\text{\textbackslash\textbackslash abacus//} &= [+0.5, +0.9, -1.3, \ldots] \\
&\ldots \\
\text{\textbackslash\textbackslash zyzzyva//} &= [-0.1, +0.8, -0.4, \ldots].
\end{aligned}
$$



Word Embedding

- the word embedding vectors have additional properties beyond mere proximity for similar words
- the **vector difference** B – A seems to **encode relationship**.

Question:

"Athens is to Greece as Oslo is to [what]?"

Answer:

D = C + (B – A)

A	B	C	D = C + (B – A)	Relationship
Athens	Greece	Oslo	Norway	*Capital*
Astana	Kazakhstan	Harare	Zimbabwe	*Capital*
Angola	kwanza	Iran	rial	*Currency*
copper	Cu	gold	Au	*Atomic Symbol*
Microsoft	Windows	Google	Android	*Operating System*
New York	New York Times	Baltimore	Baltimore Sun	*Newspaper*
Berlusconi	Silvio	Obama	Barack	*First name*
Switzerland	Swiss	Cambodia	Cambodian	*Nationality*
Einstein	scientist	Picasso	painter	*Occupation*
brother	sister	grandson	granddaughter	*Family Relation*
Chicago	Illinois	Stockton	California	*State*
possibly	impossibly	ethical	unethical	*Negative*
mouse	mice	dollar	dollars	*Plural*
easy	easiest	lucky	luckiest	*Superlative*
walking	walked	swimming	swam	*Past tense*

Word Embedding

- Word embeddings are popular because
  - they are a good representation for downstream language tasks (such as question answering or translation or summarization)
- It is possible to use **generic pretrained vectors**, obtained from any of several suppliers, for one's particular NLP task.
  - WORD2VEC, GloVe (Global Vectors), and FASTTEXT
- We can also train word embedding for a specific task on a carefully selected corpus

Word Embedding

- Context matters!

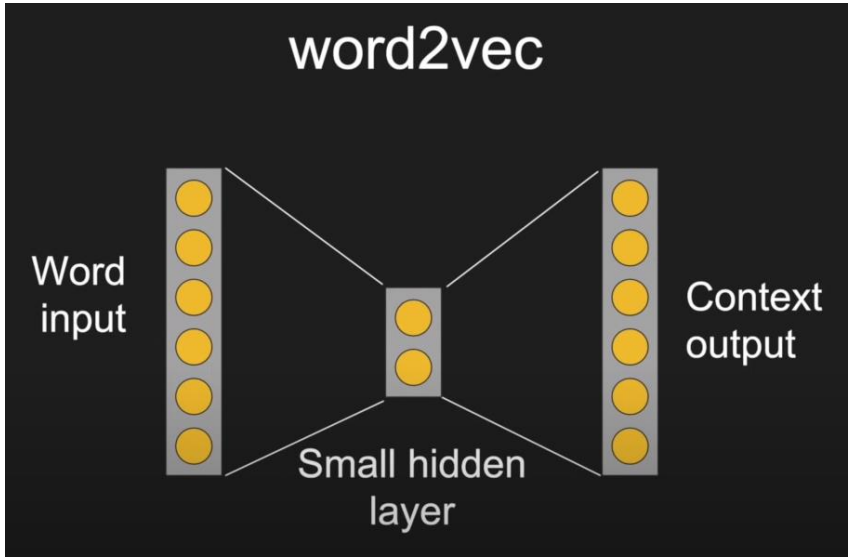
Alice is **swimming** against the current.

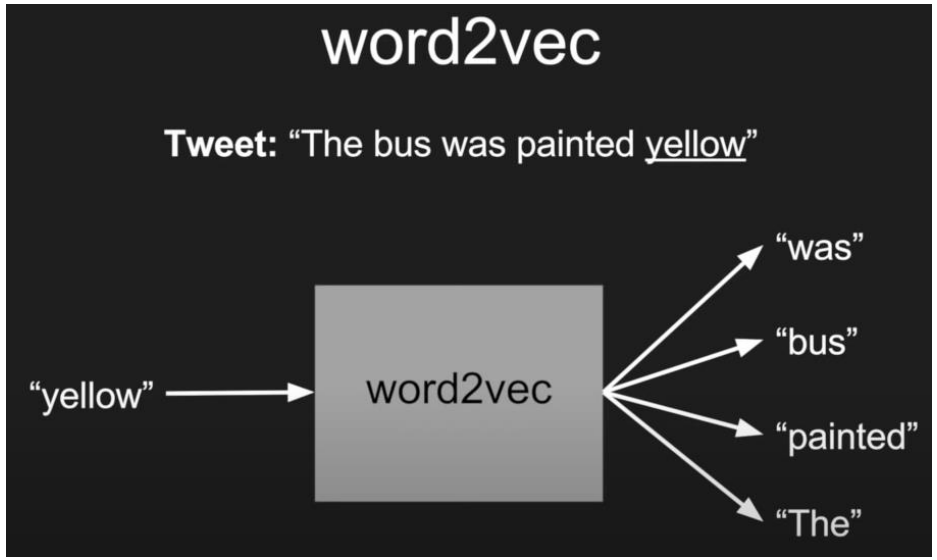
The current **version** of the **file** is in the cloud.

Word Embedding

- Context matters!

"The bus was painted yellow"

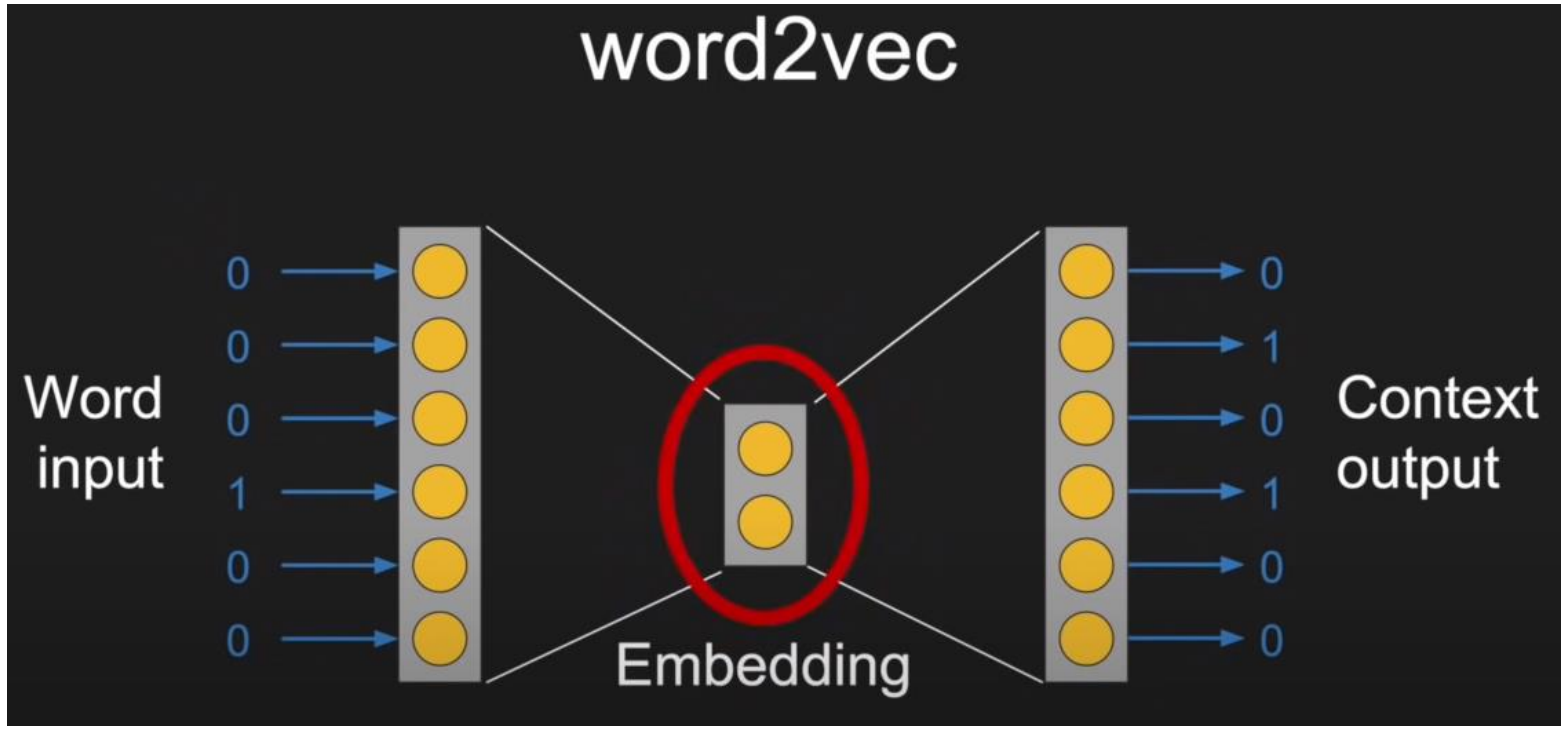




Word to Context

Word Embedding

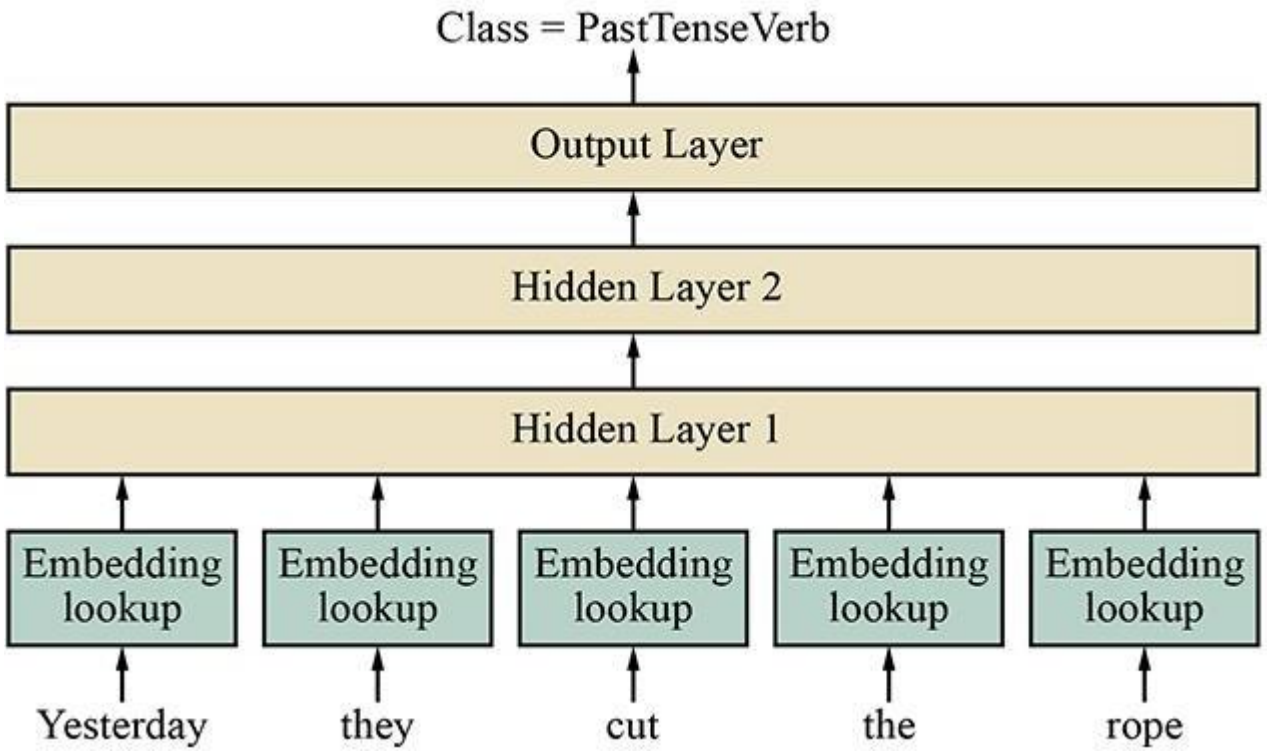
- word2vector



Or POS tags

Word Embedding

- Given a corpus of sentences with POS tags, we learn the parameters for the word embeddings and the POS tagger simultaneously.



- Feedforward part-of-speech tagging model. This model takes a 5-word window as input and predicts the tag of the word in the middle—here, *cut*.

- The parameter values for the word embeddings and for the three layers are all learned simultaneously during training.

Recurrent Neural Networks for NLP

- language consists of an ordered **sequence** of words in which the **context** of surrounding words is important
  - "Eduardo told me that Miguel was very sick so I took **him** to the hospital,"
- How to create a language model with **sufficient context**?
  - We cannot afford to increase the fixed window size
  - A feedforward network is not suitable
- Recurrent neural network (RNN) is designed to process time-series data, one datum at a time

Recurrent Neural Networks (RNNs)

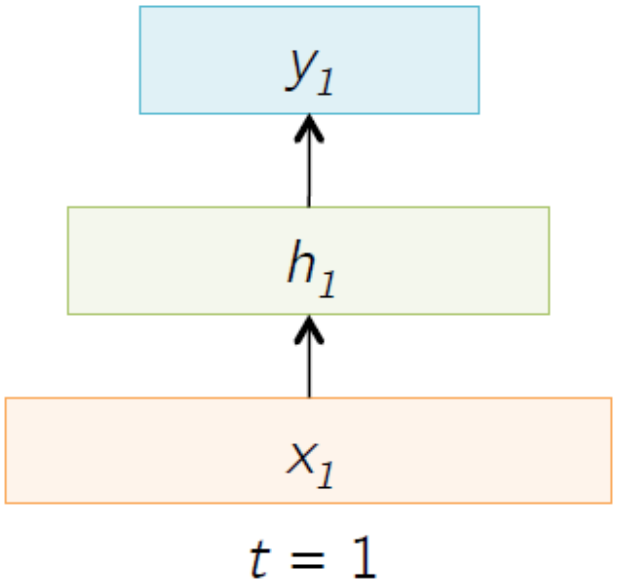
Motivation

- **Not all problems can be converted into one with fixed length inputs and outputs**!

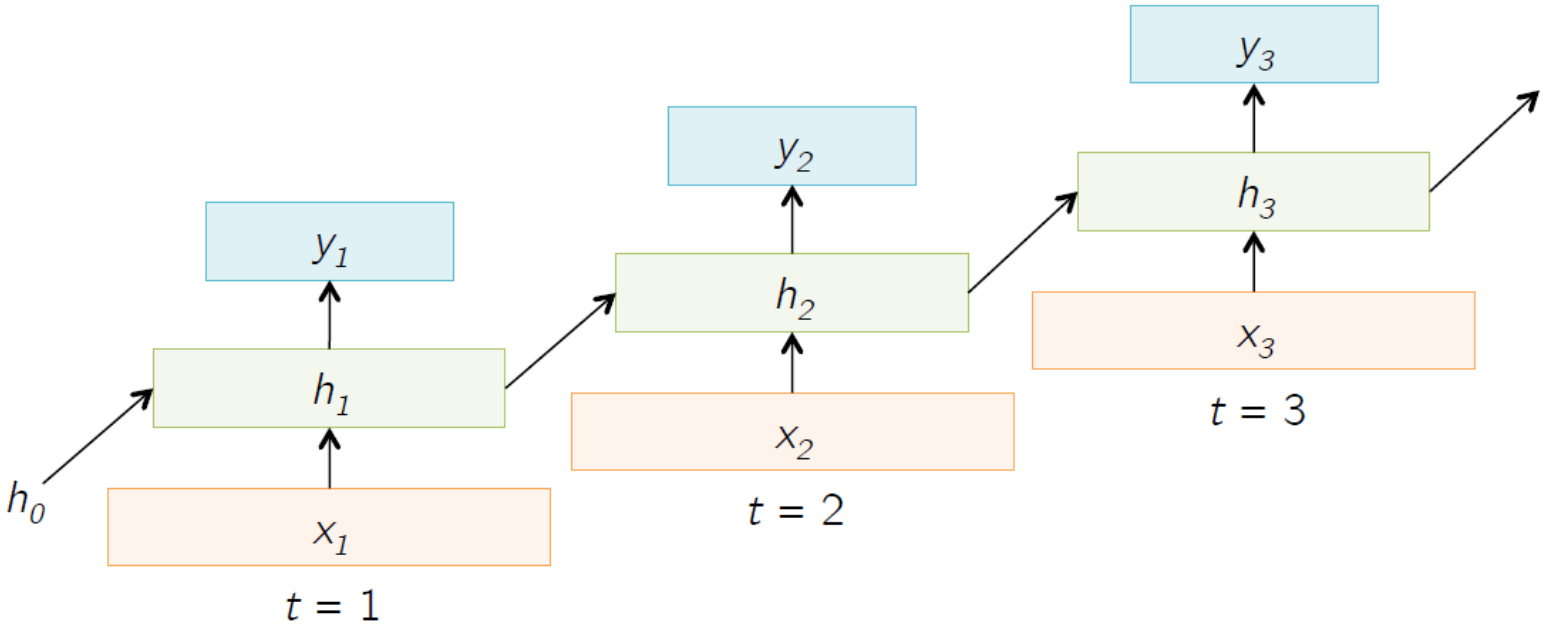
- Problems such as Speech Recognition or Time-series Prediction require a system to store and use context information
  - Simple case: Output YES if the number of 1s is even, else NO.
  - 1000010101 – YES, 100011 – NO, …

- Hard/Impossible to choose a fixed context window
- There can always be a new sample longer than anything seen!

Recurrent Neural Networks (RNNs)



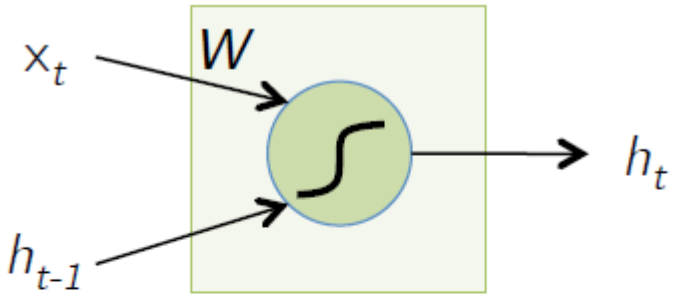
A Sample Feed-forward Network (e.g. MLP)



A Sample RNN

Recurrent Neural Networks (RNNs)

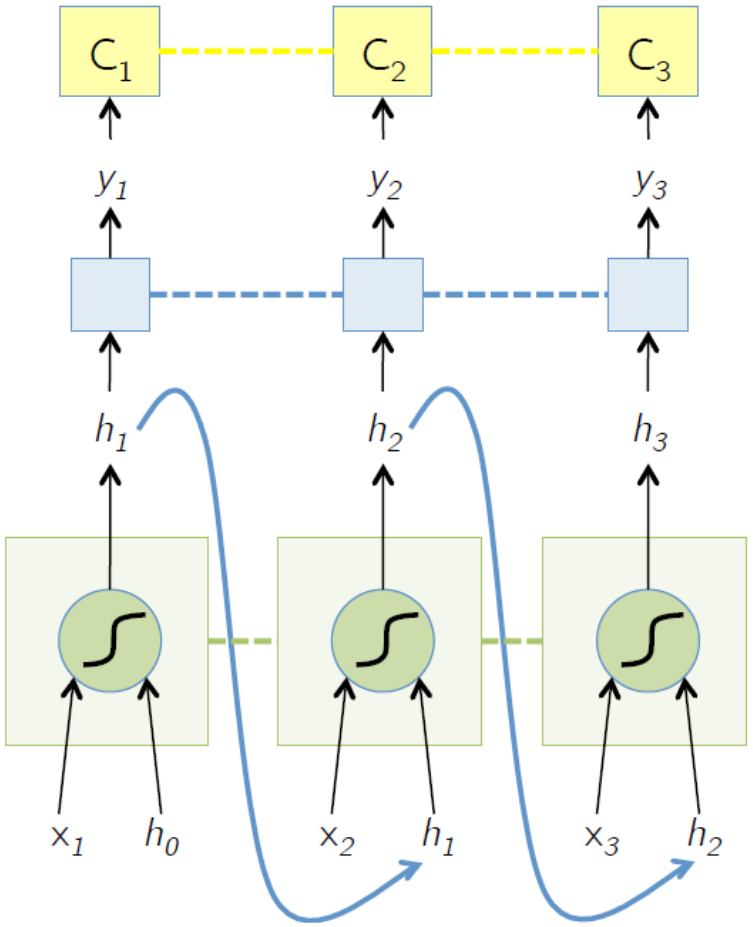
The Vanilla RNN Cell



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

Recurrent Neural Networks (RNNs)

The Vanilla RNN Forward



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$y_t = \mathrm{F}(h_t)$$

$$C_t = \mathrm{Loss}(y_t, \mathrm{GT}_t)$$

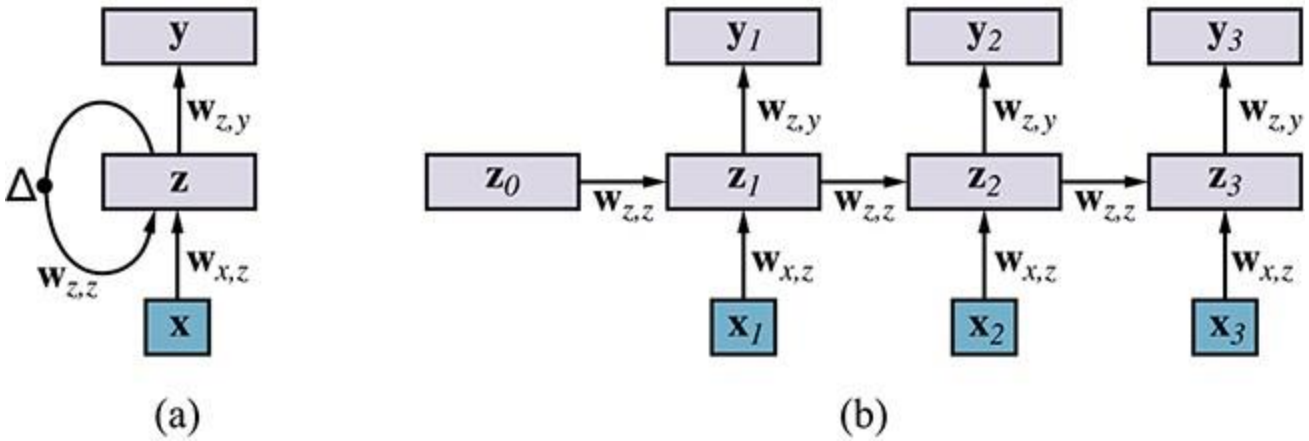
------ indicates shared weights

Recurrent Neural Networks (RNNs)

- Note that the weights are shared over time
- Essentially, copies of the RNN cell are made over time (unrolling/unfolding), with different inputs at different time steps

Recurrent Neural Networks for NLP

- RNNs can be used for processing language, one word at a time.



Note the notation z is same as h in the previous slides

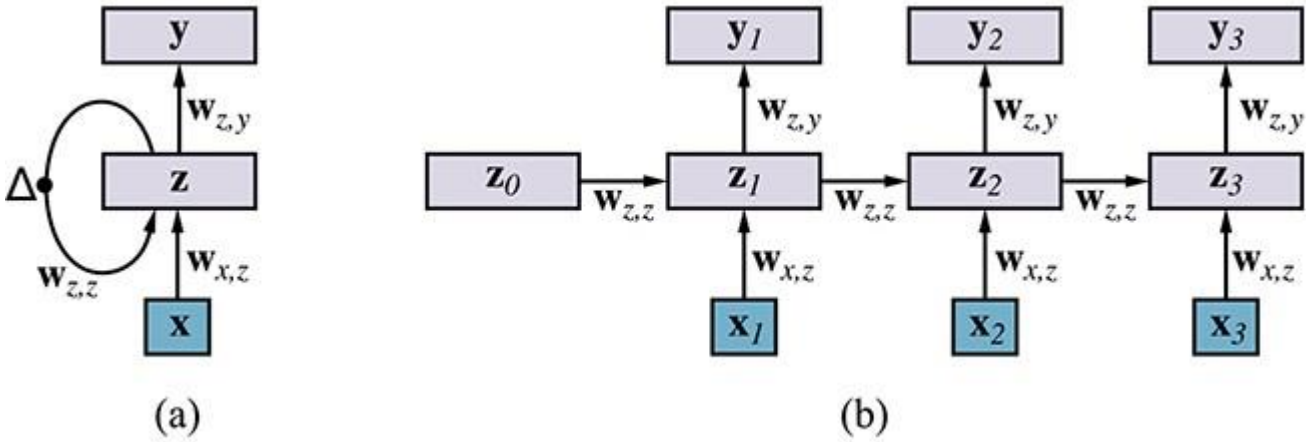
(a) Schematic diagram of an RNN where the hidden layer $\mathbf{z}_t$ has **recurrent** connections; the Δ symbol indicates a delay. Each input **x** is the word embedding vector of the next word in the sentence. Each output y is the output for that time step.

(b) The same network unrolled over three timesteps to create a feedforward network. Note that the weights are **shared** across all timesteps.

(c) We are interested in doing multiclass classification: the **classes** are the words of the vocabulary. Thus the output $\boldsymbol{y}_t$ will be a softmax probability distribution over the possible values of the next word in the sentence.

Recurrent Neural Networks for NLP

- We are training the model to **predict the next word**, and expecting that in order to do so it will use the hidden layer to represent useful information.
- Once the model has been trained, we can use it to **generate random text or for other NLP tasks**.



Marry, and will, my lord, to weep in such a one were prettiest;
Yet now I was adopted heir
Of the world's lamentable day,
To watch the next way with his father with his face?

Recurrent Neural Networks for NLP

Sentiment Classification

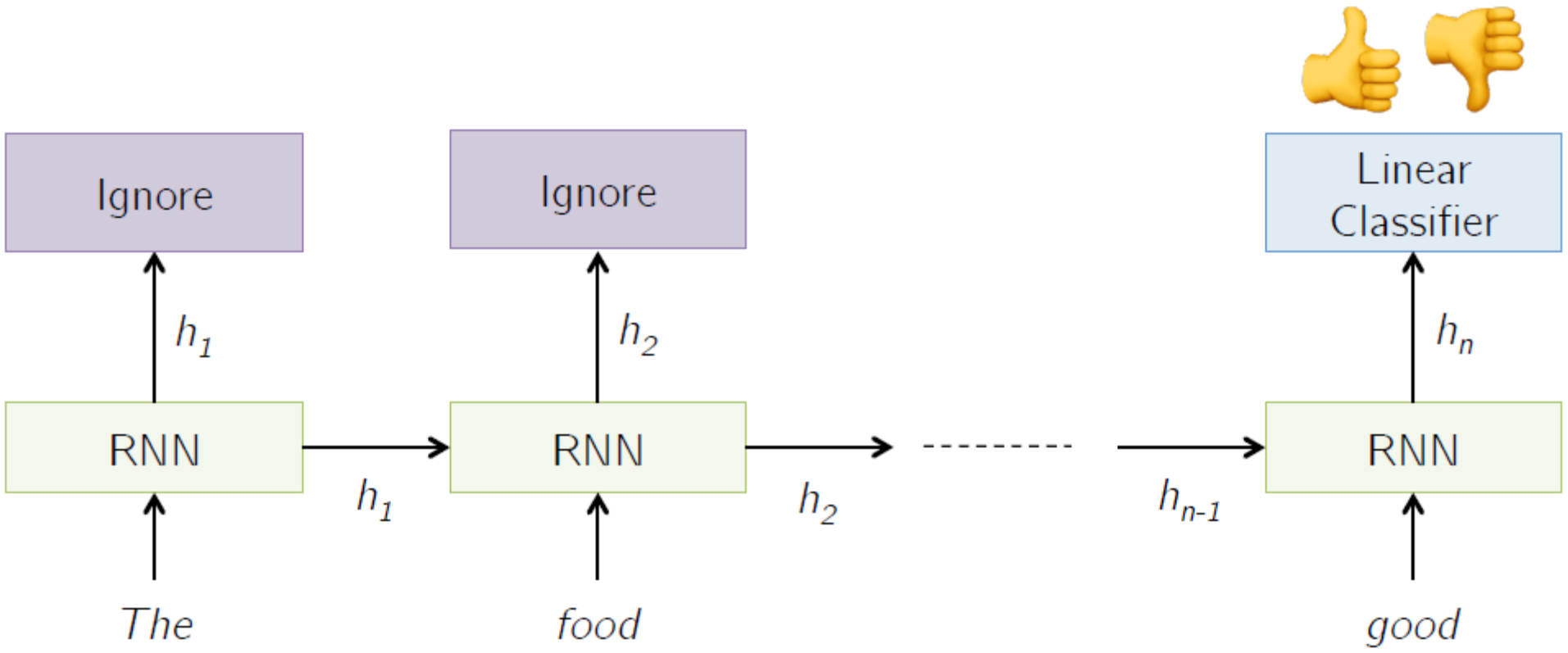
- Classify a restaurant review OR movie review as positive or negative

- **Inputs:** Multiple words, one or more sentences!
- **Outputs:** Positive / Negative classification

- Examples
  - “The food was really good”
  - “The chicken crossed the road because it was uncooked”

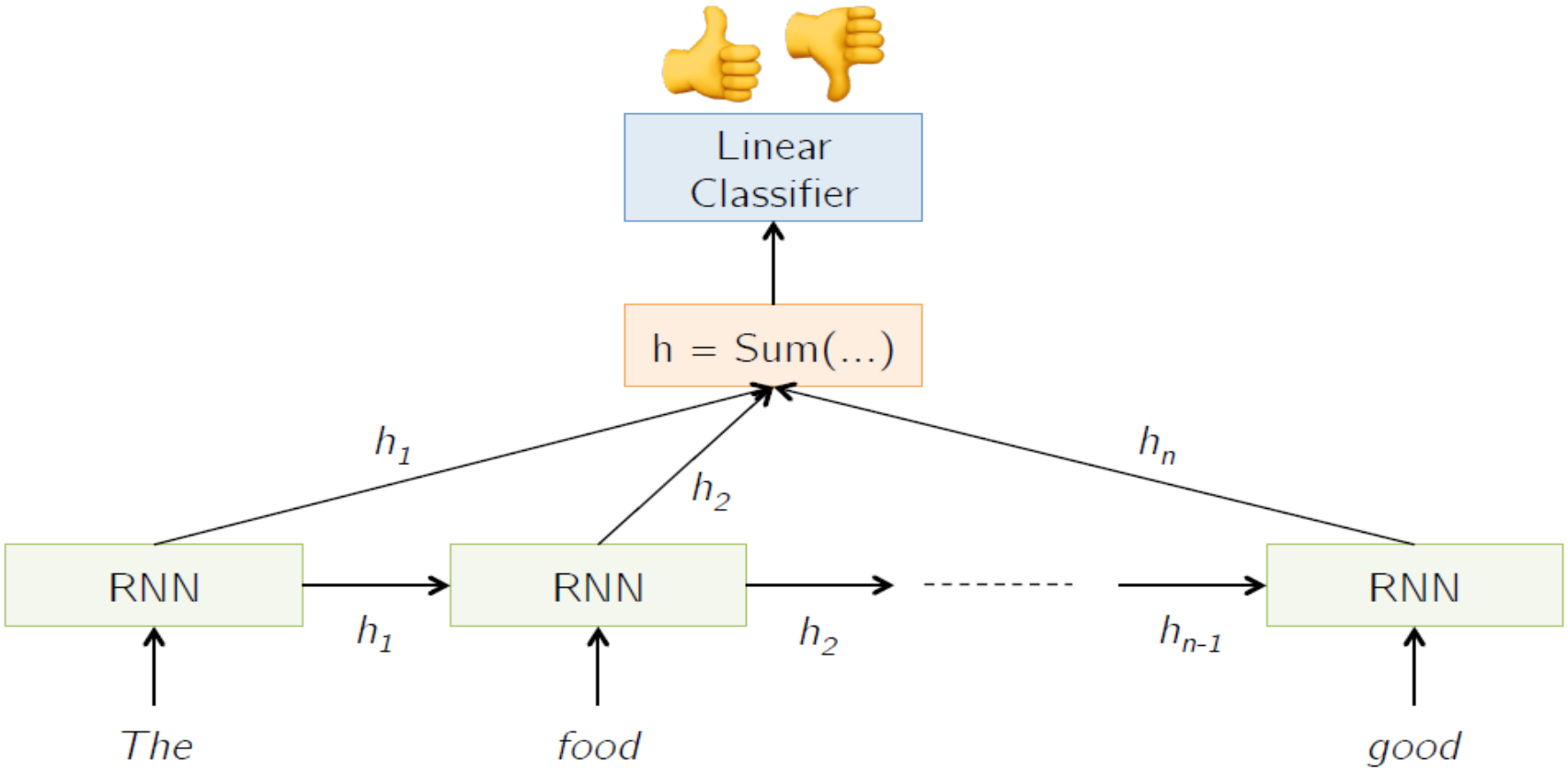
Recurrent Neural Networks for NLP

Sentiment Classification



Recurrent Neural Networks for NLP

Sentiment Classification

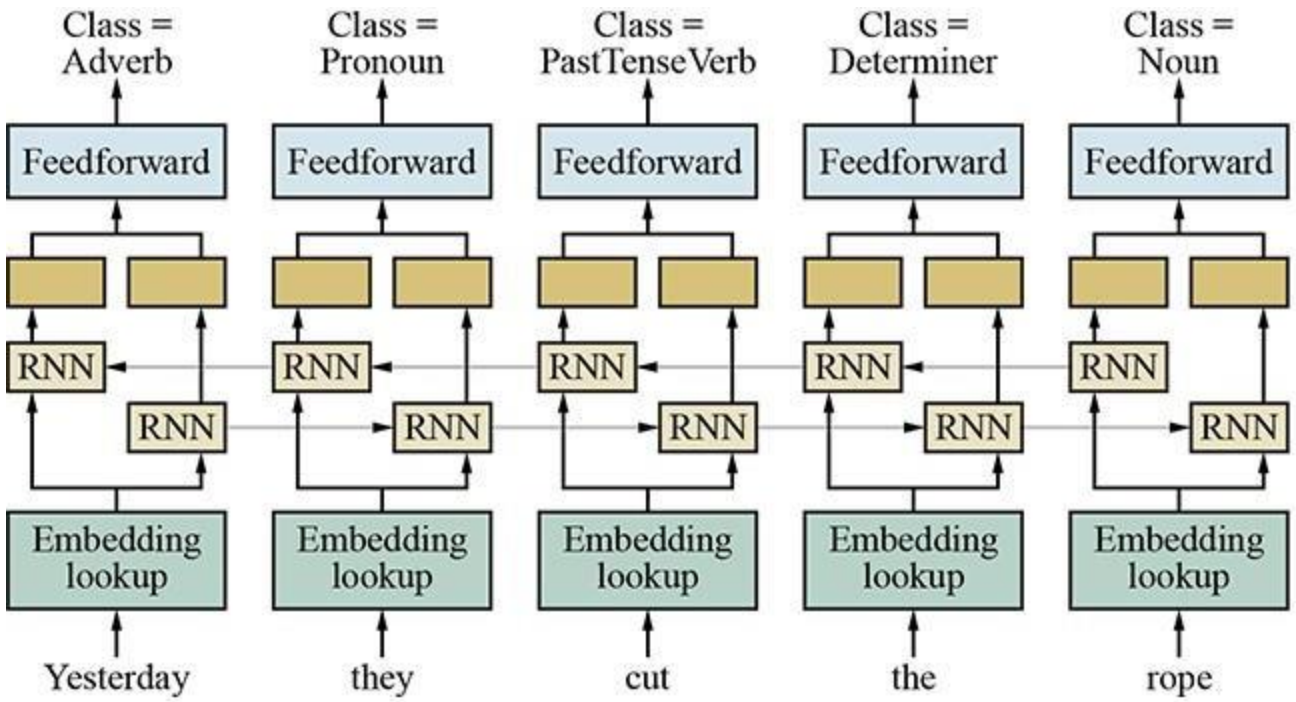


http://deeplearning.net/tutorial/lstm.html

Recurrent Neural Networks for NLP

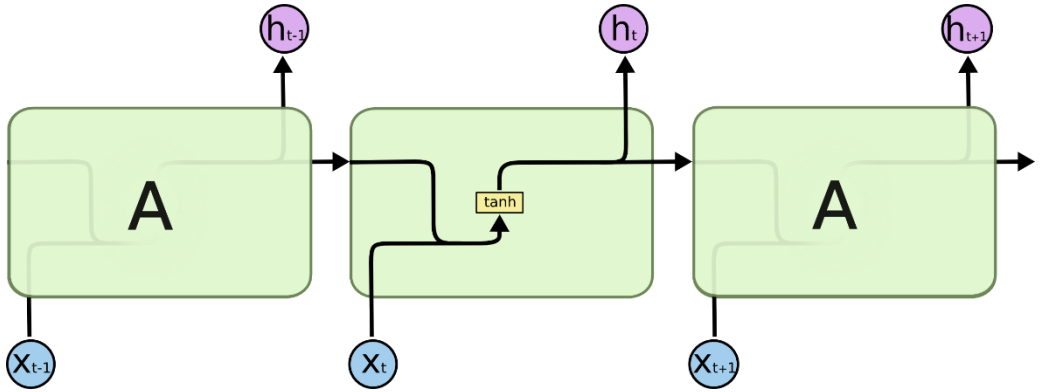
- We can use RNNs for other language tasks, say POS tagging
  - the output will be a softmax distribution over POS tags
  - the training data will require labels—part of speech tags
  - We should not limit ourselves to looking at only the previous words

A **bidirectional** RNN network for POS tagging.

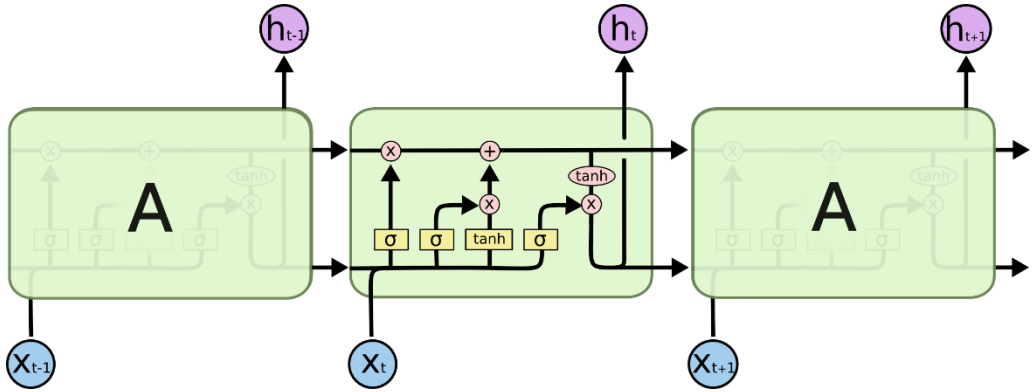


Recurrent Neural Networks for NLP

- RNN: playing the game of telephone
- Long short-term memory (LSTM) model
  - A kind of RNN with **gating units** that don't suffer from the problem of imperfectly reproducing a message from one time step to the next.
  - an LSTM can choose to **remember** some parts of the input, copying it over to the next timestep, and to **forget** other parts



The repeating module in a standard RNN contains a single layer.

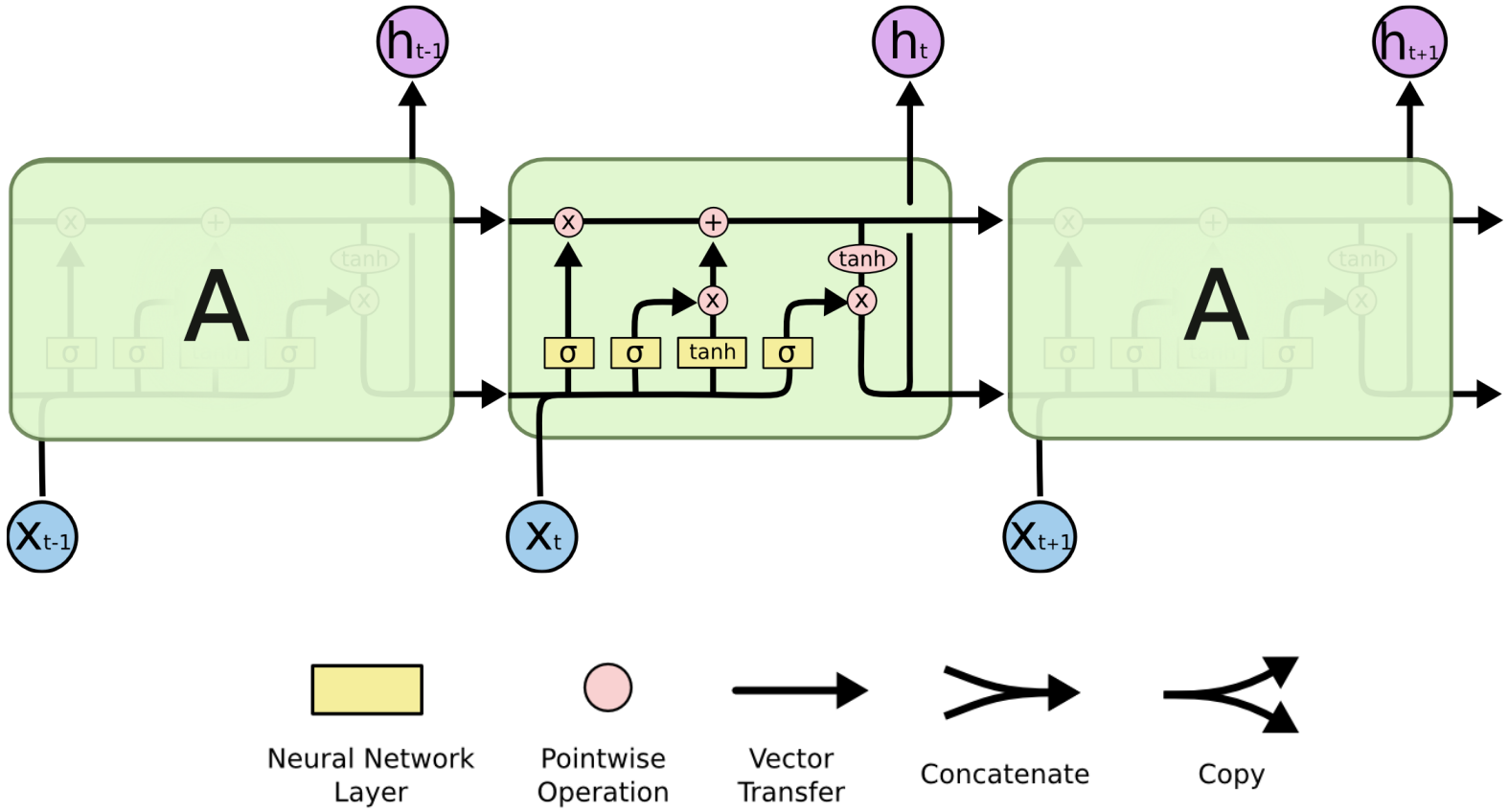


The repeating module in an LSTM contains four interacting layers.

More details are at https://colah.github.io/posts/2015-08-Understanding-LSTMs/

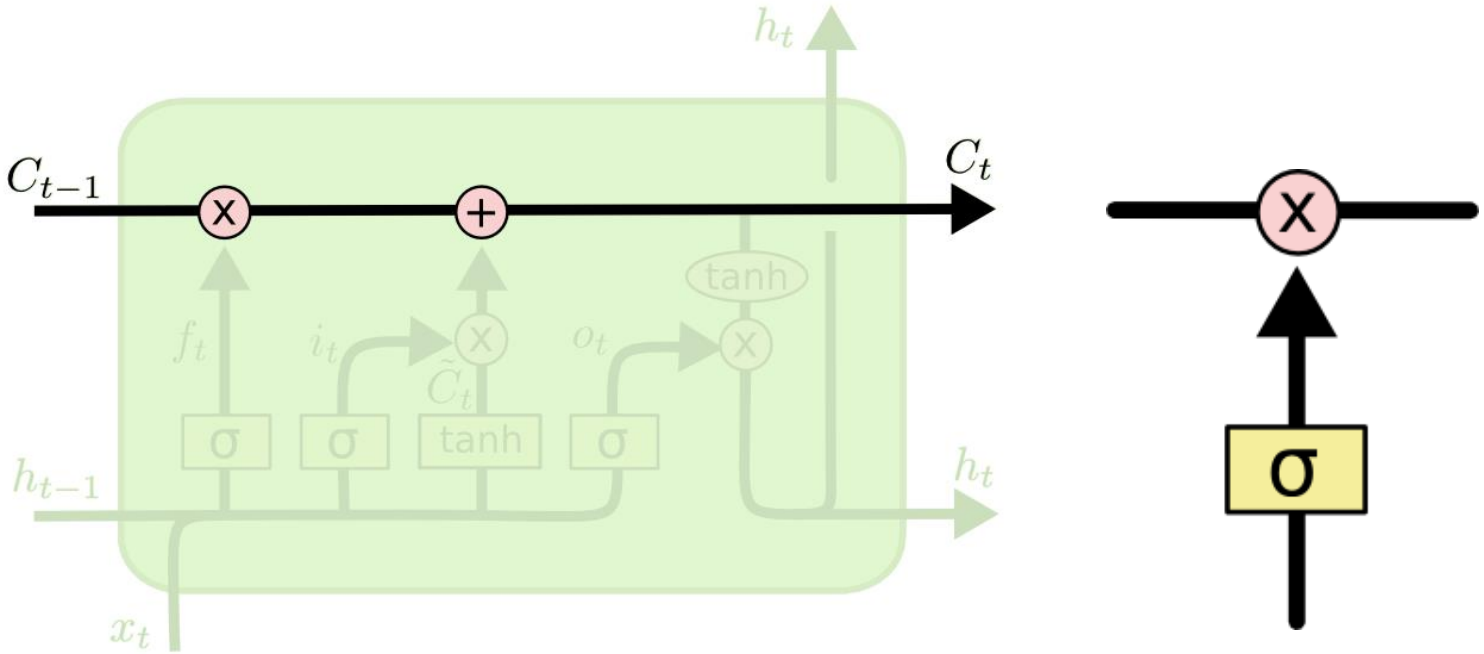
Long Sort-Term Memory (LSTM)

A Standard LSTM Layer



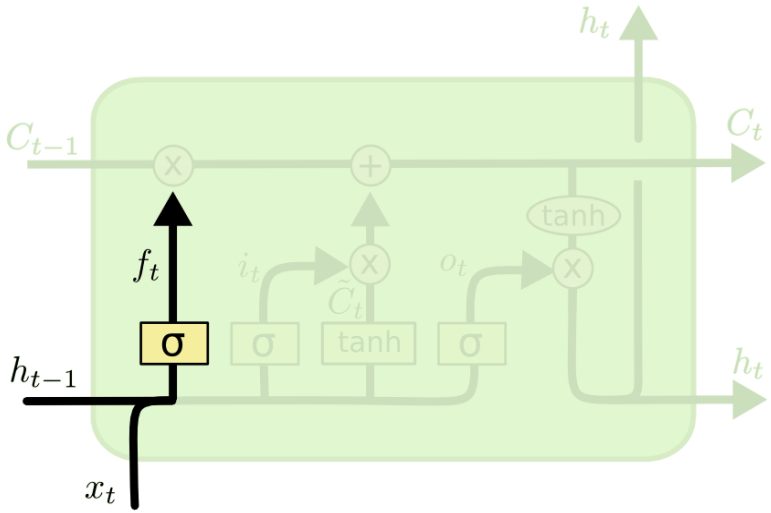
Code Idea Behind LSTMs

Cell state – remove and add information



Step-by-Step LSTM Walk Through

Deciding what information to throw away



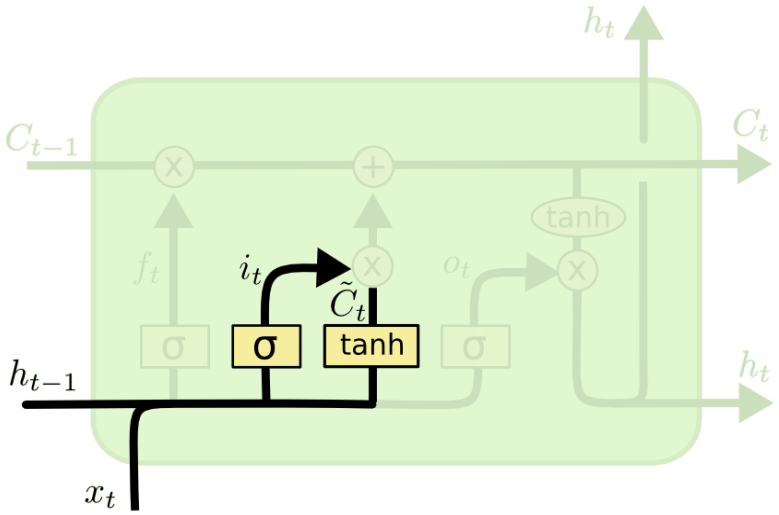
Forget gate layer

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

$f_t \in [0,1]$

Step-by-Step LSTM Walk Through

Deciding what information to store



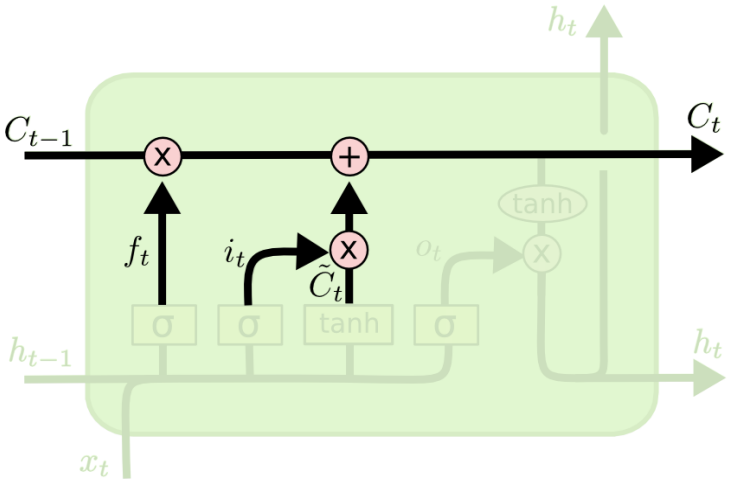
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

Input gate layer

Step-by-Step LSTM Walk Through

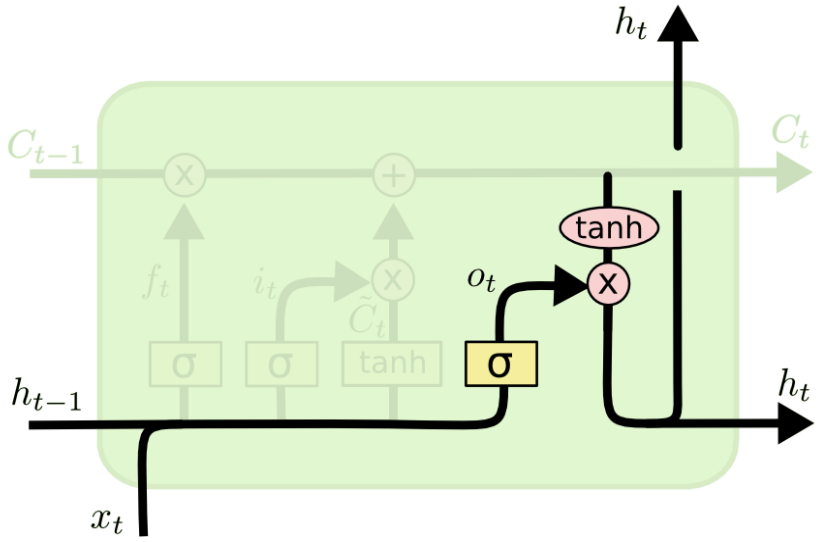
Updating the old state C_{t-1} into new state C_t



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Step-by-Step LSTM Walk Through

Deciding what to output



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

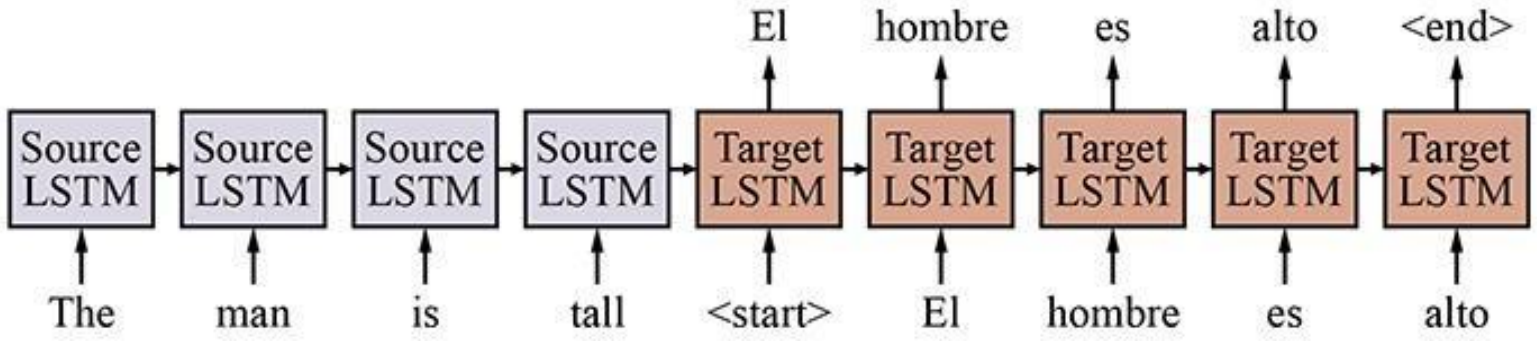
$$h_t = o_t * \tanh\left(C_t\right)$$

Sequence-to-Sequence Models

- Machine translation (MT)
  - Translate a sentence from a source language to a target language
  - It is not a simple tagging task (no one-to-one correspondence!)
  - The generation of each target word is conditional on
    1. The entire source sentence and
    2. All previously generated target words.
- We use two RNNs, one for the source and one for the target.
  - Run the source RNN over the source sentence and use the final hidden state from the source RNN as the initial hidden state for the target RNN.
  - This way, each target word is implicitly conditioned on both the entire source sentence and the previous target words.

Sequence-to-Sequence Models

- Basic sequence-to-sequence models were a significant breakthrough in NLP and MT specifically.

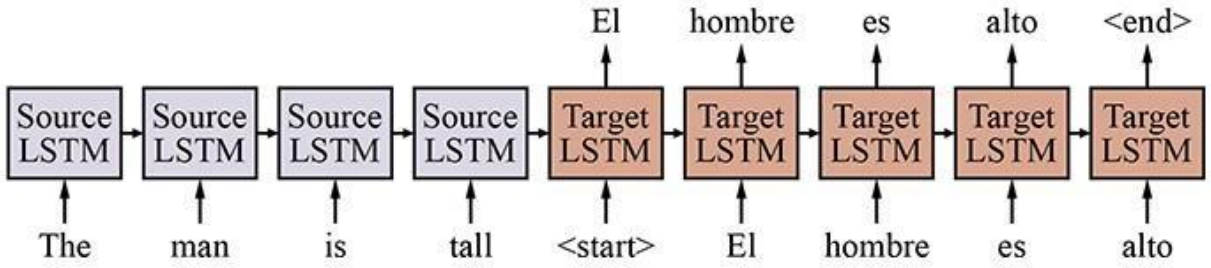


Basic sequence-to-sequence model.

1. Each block represents one LSTM timestep. (For simplicity, the embedding and output layers are not shown.)
2. On successive steps we feed the network the words of the source sentence “The man is tall,” followed by the <start> tag to indicate that the network should start producing the target sentence.
3. The final hidden state at the end of the source sentence is used as the hidden state for the start of the target sentence.
4. After that, each target sentence word at time t is used as input at time t+1, until the network produces the <end> tag to indicate that sentence generation is finished.

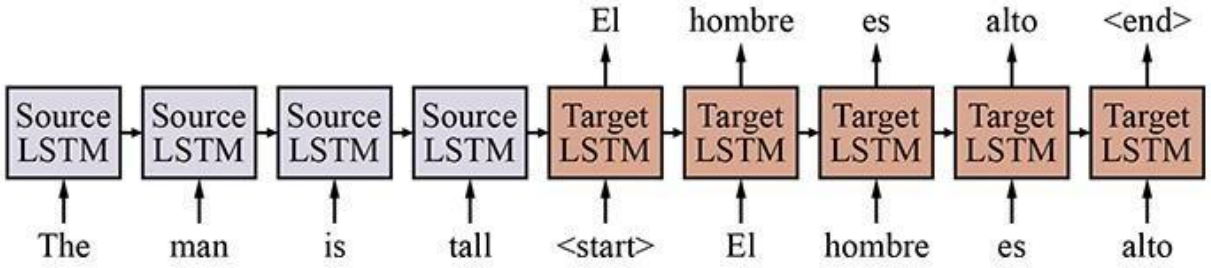
Attention

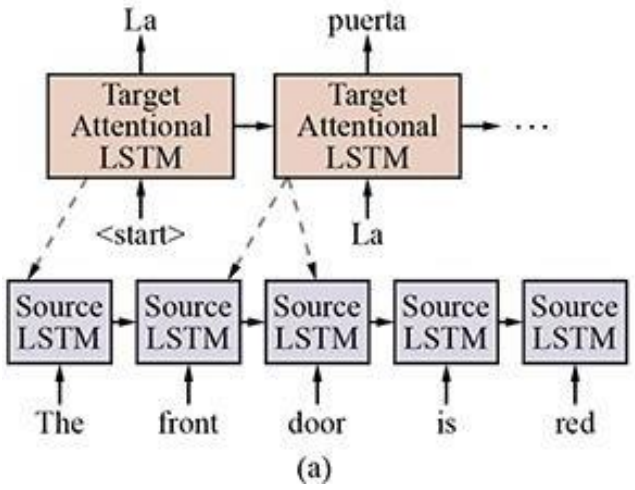
- Sequence-to-sequence model
  - Nearby context bias
  - Fixed context size limit

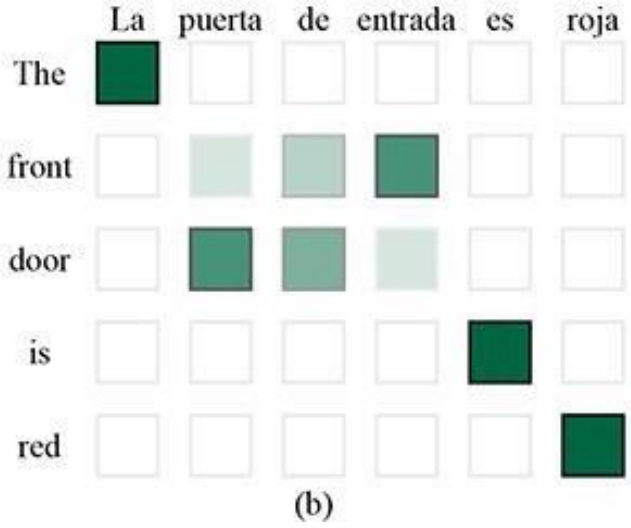


- What if the target RNN were conditioned on all of the hidden vectors from the source RNN, rather than just the last one?
  - But it is likely that only a small part of the source is actually relevant to each target word.
- A neural network component called "**attention**"
  - Create a "context-based summarization" of the source sentence into a fixed dimensional representation.

Attention







Attentional sequence-to-sequence model for English-to-Spanish translation. The dashed lines represent attention.

Example of attention probability matrix for a bilingual sentence pair, with darker boxes representing higher values of attention. The attention probabilities sum to one over each column.

Self-attention

- In the last slide, attention was applied from the target RNN to the source RNN.
- **Self-attention** extends this mechanism so that each sequence of hidden states also attends to itself
  - the source to the source, and the target to the target.
  - This allows the model to additionally capture long-distance (and nearby) context within each sequence.
- Self-attention is an important component of the **transformer** model.

Pretraining and Transfer Learning

- We use a large amount of shared general domain language data to train an initial version of an NLP model.
- From there, we can use a smaller amount of domain-specific data (perhaps including some labeled data) to refine the model.
- The refined model can learn the information specific to the new domain.
- Pretrained word embeddings
- Pretrained contextual representations

Pretraining and Transfer Learning

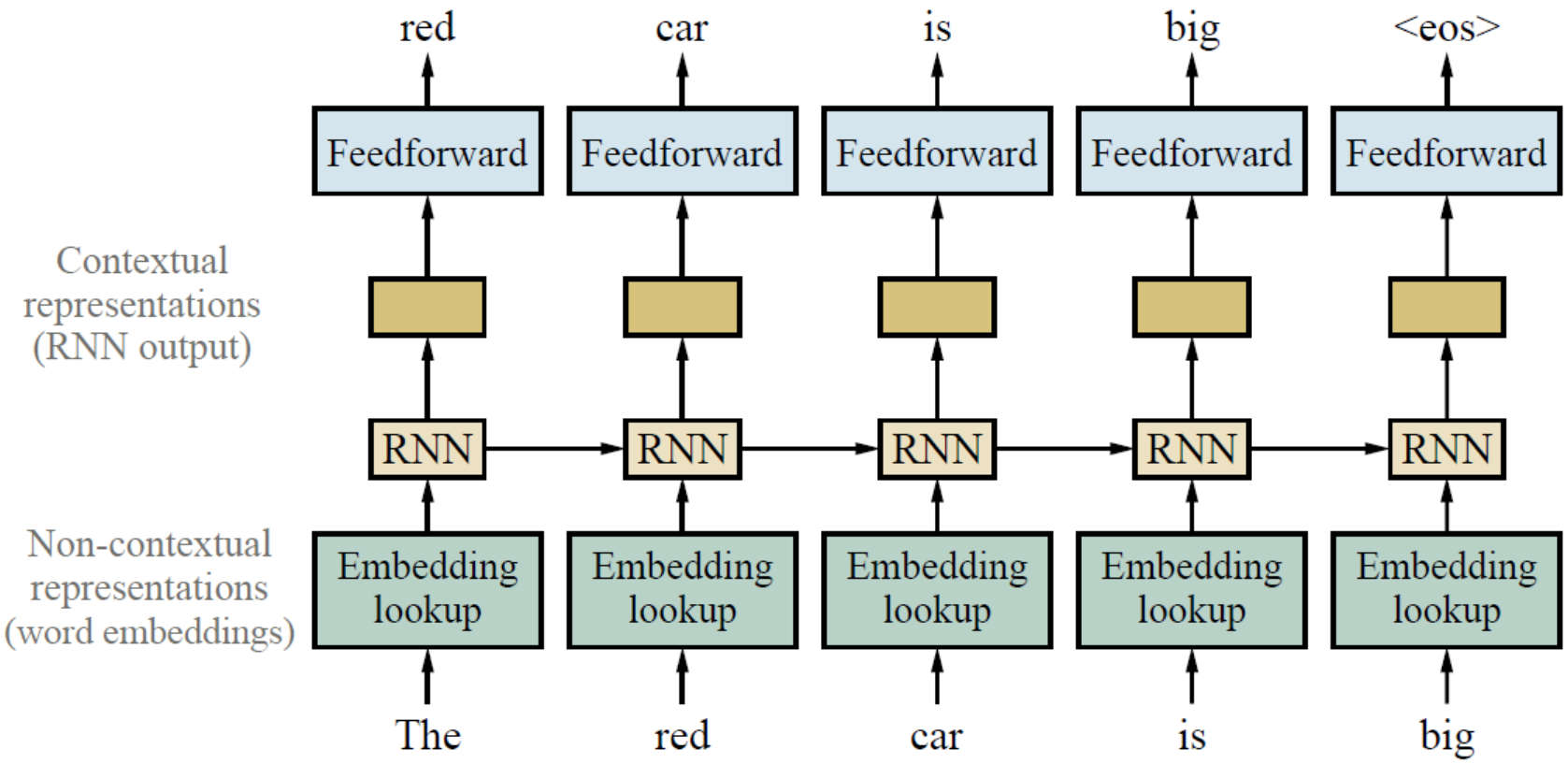
Pretrained word embeddings

GloVe (Global Vectors) model

- specific model for word embeddings
- relationship between two words can best be captured by comparing them both to other words
- starts by gathering counts of how many times each word appears within a window of another word.
- choose window size (perhaps 5 words) and let $X_{i\,j}$ be the number of times that words *i* and *j* co-occur within a window, and let X_i be the number of times word *i* co-occurs with any other word.
- Let $P_{i\,j} = X_{i\,j}/X_i$ be the probability that word *j* appears in the context of word *i*. let $\mathbf{E}_i$ be the word embedding for word *i*.
- **the dot product of two word vectors is equal to the log probability of their co-occurrence.**

Pretraining and Transfer Learning

Pretrained contextual representations



Training contextual representations using a left-to-right language model.

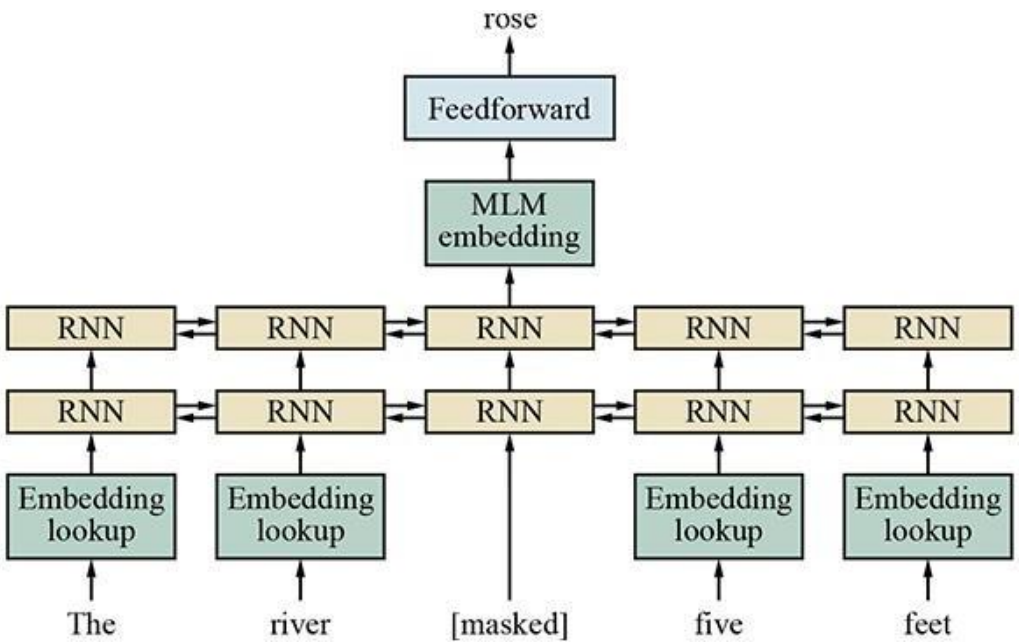
Pretraining and Transfer Learning

Masked language models

- **Masked language model** (**MLM**) are trained by masking (hiding) individual words in the input and asking the model to predict the masked words.
- For this task, one can use a deep bidirectional RNN or transformer on top of the masked sentence.
- The final hidden vectors that correspond to the masked tokens are then used to predict the words that were masked
- During training a single sentence can be used multiple times with different words masked out.
  - requires no labeled data: the sentence provides its own label for the masked word.

Pretraining and Transfer Learning

- A masked language model (MLM)
  - MLMs are trained by masking (hiding) individual words in the input and asking the model to predict the masked words.



Masked language modeling: pretrain a **bidirectional** model—for example, a multilayer RNN—by masking input words and **predicting only those masked words**.

Overview of NLP

Unstructured Text

Preprocessing

- Tokenization
- Stop word removal
- Stemming
- Lemmatization
- Part-of-speech tagging

Language Modeling

- Word-embedding
- Language Modeling

NLP Tasks

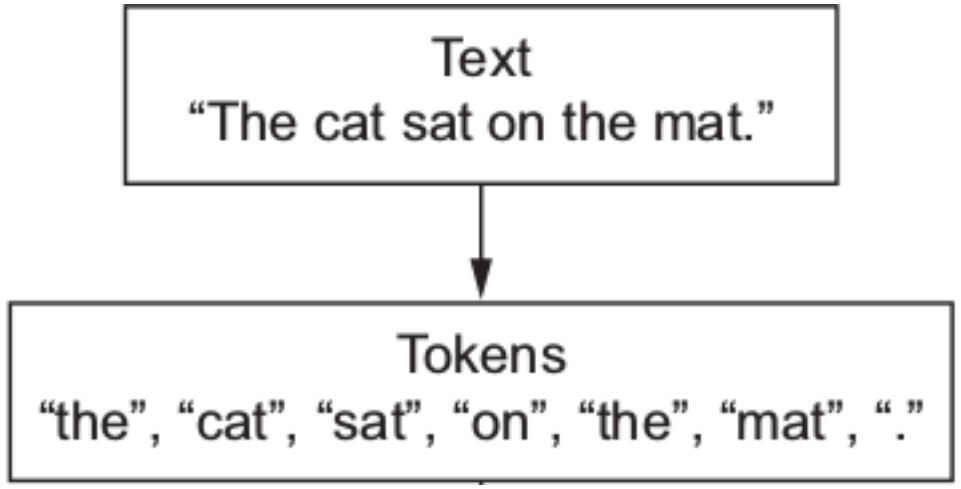
- Summarization
- Classification
- Sentiment Analysis
- Named Entity Recognition
- Topic Modeling
- Text Generation

NLP Applications

- Translation
- Chatbots
- Question Answering
- Information Retrieval
- Procedure Compliance
- NLP in Health
- ....

Preprocessing - Tokenization

To break text down into a smaller units (e.g. words, sentences). Each of the smaller unit is called tokens



The tokens could be words, numbers or punctuation marks. In tokenization, smaller units are created by locating word boundaries.

Preprocessing – Stop Word Removal

- The words which are generally filtered out before processing a text are called "**stop words**".

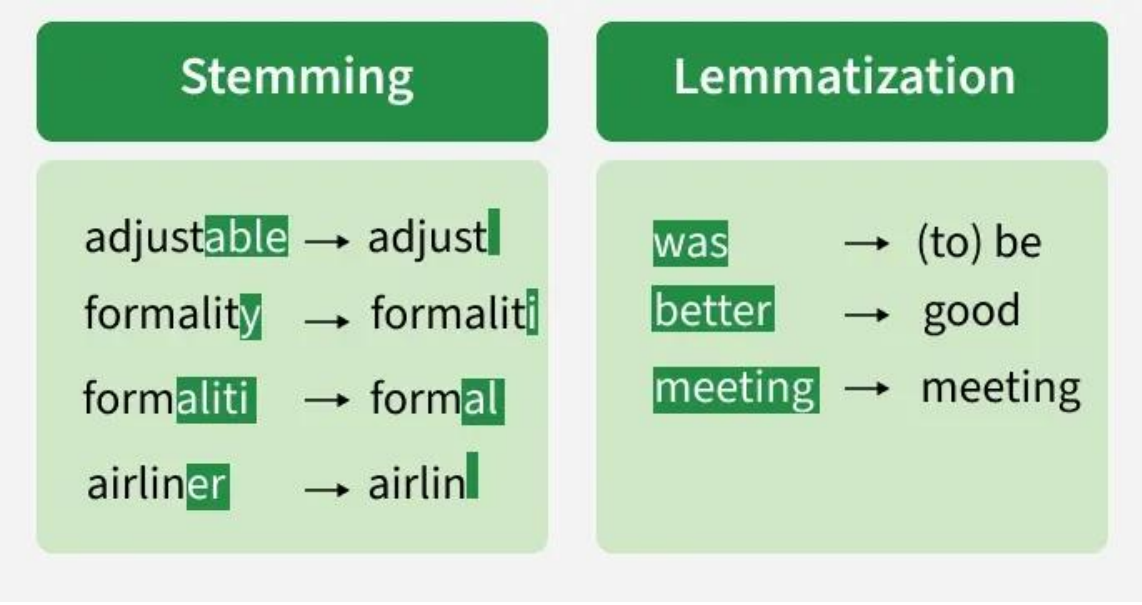
- These are actually the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc) and does not add much information to the text.
  - Examples: **"the", "a", "an", "so", "what"**

- *Definition of stop words is application dependent*

Preprocessing – Stemming and Lemmatization

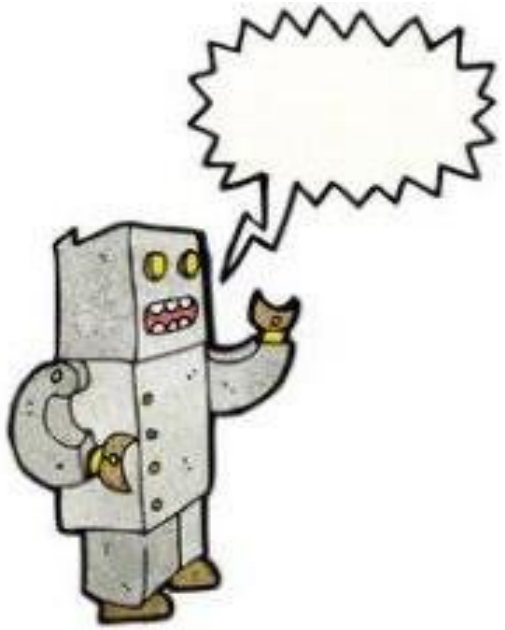
- The goal of both stemming and lemmatization is to reduce inflectional forms and sometimes derivationally related forms of a word to a common base form.
  car, cars, car's, cars' ➔ car
  am, are, is ➔be;
- **Stemming** is the process of reducing words to their word stem. A "**stem**" is the part of a word that remains after the removal of all affixes.
  - the stem for the words "touched", "touch." and "touching" is "touch".

- **Lemmatization** is the process to return to the base or dictionary form of a word, which is known as "**lemma"**



Natural Language Processing

- Deep Learning for Natural Language Processing
- Word Embedding
- Recurrent Neural Networks for NLP
- Sequence-to-Sequence Models
- Attention & Self-attention
- Pretraining and Transfer Learning



Acknowledgement

The lecture slides are based on the materials from ai.Berkey.edu

Questions?

UNIVERSITY
OF WOLLONGONG
AUSTRALIA